BÀI TẬP THỰC HÀNH

FEATURE ENGINEERING WITH PYSPARK

1. Mục tiêu thực hành

- Đọc dữ liệu CSV bằng PySpark và kiểm tra dữ liệu sau khi load.
- Thực hiện EDA bằng describe(), correlation và sampling.
- Làm sạch dữ liệu: drop cột, text filtering, duplicate và outlier.
- Thực hiện Min-Max scaling, Z-score và Log scaling.
- Đánh giá và xử lý missing values.
- Join thêm dữ liệu bên ngoài bằng DataFrame API và Spark SQL.
- Tạo feature mới bằng cộng, chia, tỷ lệ và chênh lệch.
- Tạo time features và lag features.
- Trích xuất feature từ trường text như ROOF.
- Binarize, bucketize và one-hot encode dữ liệu.
- Tạo feature vector bằng VectorAssembler.
- Huấn luyện RandomForestRegressor và đánh giá bằng RMSE, MAE, R^2.
- Phân tích feature importance và lưu/tải model.

2. Chuẩn bị

- Python 3.x.
- Jupyter Notebook/JupyterLab hoặc VS Code.
- PySpark.
- Notebook TH12_Feature_Engineering_with_PySpark.ipynb.
- Dataset 2017_StPaul_MN_Real_Estate.csv.

```
pip install pyspark pandas matplotlib
```

3. Nội dung thực hành

Khởi tạo SparkSession

Spark thay đổi nhanh, vì vậy nên kiểm tra phiên bản Spark và Python trước khi thực hành.

```
from pyspark.sql import SparkSession
import sys

spark = (
    SparkSession.builder
    .appName("FeatureEngineeringWithPySpark")
    .master("local[*]")
    .getOrCreate()
)
```

```
print("Spark version :", spark.version)
print("Python version:", sys.version)
```

Đọc dữ liệu và kiểm tra sau khi load

```
df = (
    spark.read
    .option("header", True)
    .option("inferSchema", True)
    .csv("2017_StPaul_MN_Real_Estate.csv")
)

print("Rows   :", df.count())
print("Columns:", len(df.columns))
df.show(5, truncate=False)
```

Lưu ý: Dataset đi kèm có 5.000 dòng và 73 cột. Nếu số dòng/cột khác, cần kiểm tra lại file hoặc cách đọc dữ liệu.

1. Kiểm tra schema

```
df.printSchema()
df.dtypes[:15]
```

2. EDA và thống kê mô tả

```
df.describe(
    ["LISTPRICE", "SalesClosePrice", "LivingArea", "YEARBUILT"]
).show()
```

Ngoài describe(), có thể dùng mean, stddev, min, max, skewness, covariance và correlation.

```
from pyspark.sql import functions as F

df.select(
    F.mean("SalesClosePrice").alias("mean_price"),
    F.stddev("SalesClosePrice").alias("std_price"),
    F.skewness("SalesClosePrice").alias("skew_price")
).show()
```

Correlation

```
print(df.corr("SalesClosePrice", "YEARBUILT"))
print(df.corr("SalesClosePrice", "LivingArea"))
```

Sampling trước khi toPandas()

Khi cần vẽ bằng Matplotlib/Seaborn, phải sample trước rồi mới chuyển sang Pandas để tránh đưa toàn bộ Big Data về driver.

```
sample_df = (
    df.select("SalesClosePrice", "LivingArea")
      .sample(False, 0.2, 42)
)

pandas_sample = sample_df.toPandas()
```

2. Làm sạch dữ liệu

Các lỗi dữ liệu phổ biến gồm: ghi sai, sai định dạng, duplicate, missing, sự kiện hiếm và feature không liên quan.

Drop cột

```
cols_to_drop = [
    "StreetNumberNumeric",
    "LotSizeDimensions",
    "UNITNUMBER",
    "Class"
]

df_clean = df.drop(*[
    c for c in cols_to_drop if c in df.columns
])
```

Text filtering

```
df_clean = df_clean.where(
    (~F.col("PotentialShortSale").like("Not Disclosed"))
    | F.col("PotentialShortSale").isNull()
)
```

Duplicate

```
df_clean = df_clean.dropDuplicates()
```

3. Outlier Filtering

Quy tắc lọc trong khoảng $\mu \pm 3\sigma$.

```
row = df_clean.select(
    F.mean("SalesClosePrice").alias("mean"),
    F.stddev("SalesClosePrice").alias("std")
).first()

mean_val = row["mean"]
std_val = row["std"]

low_bound = mean_val - 3 * std_val
high_bound = mean_val + 3 * std_val

df_no_outlier = df_clean.where(
    (F.col("SalesClosePrice") >= low_bound)
    & (F.col("SalesClosePrice") <= high_bound)
)
```

Lưu ý: Dataset đi kèm có 5.000 dòng và 73 cột. Nếu số dòng/cột khác, cần kiểm tra lại file hoặc cách đọc dữ liệu.

4. Scaling và Transformation

Min-Max Scaling

```
row = df_no_outlier.select(
    F.min("DAYSONMARKET").alias("min"),
    F.max("DAYSONMARKET").alias("max")
).first()

df_scaled = df_no_outlier.withColumn(
    "scaled_days",
    (F.col("DAYSONMARKET") - F.lit(row["min"]))
    / (F.lit(row["max"]) - F.lit(row["min"]))
)
```

Standardization

```
row = df_scaled.select(
    F.mean("DAYSONMARKET").alias("mean"),
    F.stddev("DAYSONMARKET").alias("std")
).first()

df_scaled = df_scaled.withColumn(
    "z_days",
    (F.col("DAYSONMARKET") - F.lit(row["mean"]))
    / F.lit(row["std"])
)
```

Log Scaling

```
df_scaled = df_scaled.withColumn(
    "log_SalesClosePrice",
    F.log(F.col("SalesClosePrice"))
)
```

5. Missing Data

Cần phân biệt Missing Completely at Random, Missing at Random và Missing Not at Random trước khi quyết định drop hay impute.

Đếm missing

```
for c in ["ROOF", "RoomArea1", "RoomArea2", "DAYSONMARKET"]:
    if c in df_scaled.columns:
        print(
            c,
            df_scaled.where(F.col(c).isNull()).count()
        )
```

Imputation bằng mean

```
mean_dom = df_scaled.select(
    F.mean("DAYSONMARKET")
).first()[0]

df_imputed = df_scaled.fillna(
    {"DAYSONMARKET": mean_dom}
)
```

Mở rộng: báo cáo tỷ lệ missing

```
def missing_report(df):
    total = df.count()
    result = []

    for c in df.columns:
        missing = df.where(F.col(c).isNull()).count()
        result.append((c, missing, missing / total))

    return sorted(result, key=lambda x: x[2], reverse=True)
```

6. Join dữ liệu ngoài

Dữ liệu ngoài có thể tăng chất lượng dự báo nhưng cũng dễ gây data leakage. Ví dụ thêm mortgage rate theo tháng.

```
df_join = (
    df_imputed
    .withColumn("LISTDATE_DATE", F.to_date("LISTDATE"))
```

```
    .withColumn(
        "year_month",
        F.date_format("LISTDATE_DATE", "yyyy-MM")
    )
)

df_join = df_join.join(
    mortgage_df,
    on="year_month",
    how="left"
)
```

Spark SQL Join

```
df_join.createOrReplaceTempView("real_estate")
mortgage_df.createOrReplaceTempView("mortgage")

sql_join = spark.sql('''
SELECT r.*, m.mortgage_rate
FROM real_estate r
LEFT JOIN mortgage m
ON r.year_month = m.year_month
''')
```

7. Feature Generation

Các feature mới thường được tạo bằng phép nhân, cộng, chia hoặc hiệu.

```
df_feat = df_join.withColumn(
    "SQFT_TOTAL",
    F.coalesce(F.col("SQFTABOVEGROUND"), F.lit(0))
    + F.coalesce(F.col("SQFTBELOWGROUND"), F.lit(0))
)

df_feat = df_feat.withColumn(
    "LISTING_PRICE_PER_SQFT",
    F.when(
        F.col("SQFT_TOTAL") > 0,
        F.col("LISTPRICE") / F.col("SQFT_TOTAL")
    )
)

df_feat = df_feat.withColumn(
    "SALE_TO_LIST_RATIO",
    F.when(
        F.col("LISTPRICE") > 0,
        F.col("SalesClosePrice") / F.col("LISTPRICE")
    )
)
```

Lưu ý: SALE_TO_LIST_RATIO dùng trực tiếp target SalesClosePrice, nên không được đưa vào mô hình dự đoán trước khi giao dịch hoàn tất. Đây là ví dụ tốt để thảo luận data leakage.

8. Time Features

```
from pyspark.sql.functions import (
    year, month, dayofmonth, weekofyear, datediff
)

df_feat = (
```

```
    df_feat
    .withColumn("LIST_YEAR", year("LISTDATE_DATE"))
    .withColumn("LIST_MONTH", month("LISTDATE_DATE"))
    .withColumn("LIST_DAY", dayofmonth("LISTDATE_DATE"))
    .withColumn("LIST_WEEK", weekofyear("LISTDATE_DATE"))
)
```

Days on Market

```
df_feat = df_feat.withColumn(
    "OFFMARKET_DATE",
    F.to_date("offmarketdate")
)

df_feat = df_feat.withColumn(
    "DAYS_ON_MARKET_FROM_DATES",
    datediff("OFFMARKET_DATE", "LISTDATE_DATE")
)
```

9. Lag Features

```
from pyspark.sql.window import Window
from pyspark.sql.functions import lag

w = Window.orderBy("LISTDATE_DATE", "MLSID")

df_feat = df_feat.withColumn(
    "PREV_SALE_PRICE",
    lag("SalesClosePrice", 1).over(w)
)
```

Lưu ý: Lag chỉ hợp lệ nếu giá trị trước đó thực sự đã biết tại thời điểm dự đoán.

10. Trích xuất Feature từ Text

```
find_under_8 = F.col("ROOF").like("%Age 8 Years or Less%")
find_over_8 = F.col("ROOF").like("%Age Over 8 Years%")

df_feat = df_feat.withColumn(
    "old_roof",
    F.when(find_over_8, 1)
     .when(find_under_8, 0)
     .otherwise(None)
)

df_feat = df_feat.withColumn(
    "Roof_Material",
    F.split(F.col("ROOF"), ",").getItem(0)
)
```

11. Binarizing, Bucketing và One-Hot Encoding

Binarizer

```
from pyspark.ml.feature import Binarizer

binarizer = Binarizer(
    threshold=0.0,
    inputCol="Fireplaces_double",
    outputCol="Has_Fireplace"
)
```

Bucketizer

```
from pyspark.ml.feature import Bucketizer

bucketizer = Bucketizer(
    splits=[float("-inf"), 1, 2, 3, 4, float("inf")],
    inputCol="BATHSTOTAL_double",
    outputCol="Bath_Bucket"
)
```

One-Hot Encoding

```
from pyspark.ml.feature import StringIndexer, OneHotEncoder

city_indexer = StringIndexer(
    inputCol="City",
    outputCol="City_Index",
    handleInvalid="keep"
)

city_encoder = OneHotEncoder(
    inputCols=["City_Index"],
    outputCols=["City_Vec"]
)
```

12. Train/Test Split theo thời gian

Đối với dữ liệu có yếu tố thời gian, không nên random split nếu mục tiêu là mô phỏng dự báo tương lai.

- Train: các giao dịch trước thời điểm split.
- Test: các giao dịch sau thời điểm split.
- Không dùng feature chứa thông tin từ tương lai.

13. VectorAssembler

```
from pyspark.ml.feature import VectorAssembler

numeric_features = [
    "LISTPRICE",
    "OriginalListPrice",
    "LivingArea",
    "SQFTABOVEGROUND",
    "SQFT_TOTAL",
    "YEARBUILT",
    "DAYSONMARKET",
    "Bedrooms",
    "BATHSTOTAL_double",
    "Has_Fireplace",
    "LIST_MONTH",
    "mortgage_rate"
]

assembler = VectorAssembler(
    inputCols=numeric_features + ["City_Vec"],
    outputCol="features",
    handleInvalid="keep"
)
```

14. Random Forest Regression

```
from pyspark.ml.regression import RandomForestRegressor

rf = RandomForestRegressor(
    featuresCol="features",
    labelCol="SalesClosePrice",
    predictionCol="Prediction_Price",
    numTrees=100,
    maxDepth=8,
    seed=42
)

rf_model = rf.fit(train_ready)
predictions = rf_model.transform(test_ready)
```

15. Đánh giá mô hình

```
from pyspark.ml.evaluation import RegressionEvaluator

evaluator = RegressionEvaluator(
    labelCol="SalesClosePrice",
    predictionCol="Prediction_Price"
)

rmse = evaluator.evaluate(
    predictions,
    {evaluator.metricName: "rmse"}
)

mae = evaluator.evaluate(
    predictions,
    {evaluator.metricName: "mae"}
)

r2 = evaluator.evaluate(
    predictions,
    {evaluator.metricName: "r2"}
)
```

Metric	Ý nghĩa	Cách đọc
RMSE	Căn bậc hai sai số bình phương trung bình	Nhạy với lỗi lớn; càng nhỏ càng tốt
MAE	Sai số tuyệt đối trung bình	Dễ diễn giải theo đơn vị giá
R^2	Mức độ giải thích biến thiên target	Gần 1 thường tốt hơn

16. Feature Importance

Random Forest cho phép xem feature importance để hỗ trợ diễn giải mô hình.

```
importance = rf_model.featureImportances.toArray()

importance_df = pd.DataFrame({
    "feature": feature_names,
    "importance": importance
```

```
}).sort_values(
    "importance",
    ascending=False
)
```

Lưu ý: Feature importance không đồng nghĩa với quan hệ nhân quả. Các biến tương quan mạnh có thể chia sẻ importance.

17. Lưu và tải model

```
# Lưu
rf_model.write().overwrite().save(
    "rfr_real_estate_model"
)

# Tải lại
from pyspark.ml.regression import RandomForestRegressionModel

model2 = RandomForestRegressionModel.load(
    "rfr_real_estate_model"
)
```

4. Bài tập mở rộng

Bài 1. Viết hàm kiểm tra số dòng, số cột và tỷ lệ missing của target.

Bài 2. Tìm 10 feature số có correlation tuyệt đối cao nhất với SalesClosePrice.

Bài 3. So sánh lọc outlier bằng 3σ và IQR.

Bài 4. Min-Max và Z-score cho LISTPRICE, LivingArea, DAYSONMARKET.

Bài 5. Tạo bảng missing count và missing percentage cho toàn bộ cột.

Bài 6. Tạo House_Age = năm bán - YEARBUILT.

Bài 7. Tạo Rooms_per_1000Sqft và Taxes_to_ListPrice.

Bài 8. Tạo quarter, day-of-week và season từ LISTDATE.

Bài 9. Trích xuất feature từ ROOF, BASEMENT và GarageDescription.

Bài 10. So sánh LinearRegression, RandomForestRegressor và GBTRegressor.

5. Bài tập nâng cao

Xây dựng pipeline hoàn chỉnh từ dữ liệu thô đến mô hình hồi quy.

- Load và validate dữ liệu.
- EDA và kiểm tra missing/outlier.
- Làm sạch dữ liệu.
- Tạo ít nhất 5 feature mới.
- Encode categorical features.
- Tạo feature vector.
- Chia train/test hợp lý.
- So sánh ít nhất 2 mô hình.
- Báo cáo RMSE, MAE và R^2.
- Phân tích Top-10 feature quan trọng.

- Thảo luận data leakage, missing bias và high-cardinality.

6. Yêu cầu nộp bài

- Notebook đã chạy và có output.
- Dataset hoặc đường dẫn dataset.
- Bảng mô tả feature gốc và feature mới.
- Bảng so sánh mô hình.
- RMSE, MAE, R^2 trên test set.
- Top feature importance/hệ số.
- Nhận xét về missing, outlier và leakage.
- README ngắn mô tả cách chạy.

Tổng kết

Load → Validate → EDA → Clean → Engineer Features → Encode → Vectorize → Model → Evaluate → Interpret

Feature Engineering không chỉ là thao tác kỹ thuật. Việc tạo và lựa chọn feature phải xuất phát từ mục tiêu phân tích, ngữ cảnh nghiệp vụ và thông tin thật sự có sẵn tại thời điểm dự đoán.

---o0o---

(Hết)